Expense Mate
Members: Areesha 24k-3043, Burhan 24k-3056
Phase 03
Unit Test & Code Coverage Report

Test Framework: pytest 9.1.1 + pytest-cov 7.1.0 (Python 3.12.10)
Execution Date: 2026-09-24

144 Passed / 0 Failed · 100% Pass Rate · 97% Coverage

1. Executive Summary

As required by Phase 3 of the Software Construction assignment, a comprehensive automated test suite was developed and executed.

144 Total Test Cases	**144** Passed	**0** Failed	**~42.8s** Execution Time
696 Statements Analyzed	**677** Statements Covered	**19** Missing Statements	**97%** Net Project Coverage

2. Code Coverage by Module

The system is separated into three distinct layers as required by **NFR-7** (UI, Services/Business Logic, Data/Database). Code coverage across all modules:

Module / File	Stmts	Miss	Coverage %	Status	Primary Responsibilities Tested
data/budget_repo.py	7	0	**100%**	Perfect	Budget table queries & upsert operations
data/category_repo.py	8	0	**100%**	Perfect	Category retrieval and insertion
data/database.py	65	3	**95%**	Exceptional	SQLite connection, schema initialization, foreign keys
data/transaction_repo.py	29	0	**100%**	Perfect	Transaction CRUD operations & bulk insert
services/budget_calculator.py	52	1	**98%**	Exceptional	Budget limits, spent sum, remaining, % used
services/budget_service.py	38	0	**100%**	Perfect	Coordination between repos & budget calculator
services/category_service.py	13	0	**100%**	Perfect	Category validation and uniqueness

services/csv_validator.py	93	6	**94%**	Exceptional	All-or-nothing CSV parsing, column header order, row validation
services/money.py	51	0	**100%**	Perfect	Minor currency unit arithmetic, decimal parsing, negative amount rejection
services/savings_logic.py	46	0	**100%**	Perfect	Income vs. expense totals, monthly trend groupings, savings delta
services/transaction_service.py	73	7	**90%**	High	Transaction lifecycle orchestration, date range filters
services/validators.py	58	0	**100%**	Perfect	ISO date bounds (1900–2100), type validation, string sanitization
ui/charts.py	90	0	**100%**	Perfect	Matplotlib bar generation for budget and 3-series trends
ui/components.py	73	2	**97%**	Exceptional	Flash notifications, alert badges, metric cards, shared widgets
TOTAL	**696**	**19**	**97%**	**Pass**	**Comprehensive system coverage**

3. Test Suite Breakdown by Test File

The test suite is organized into modular test files mapped to individual system components:

3.1 Core Logic & Calculations (Unit Tests)

tests/test_budget_calculator.py (15 Tests)

- Month boundary calculations (get_month_range)
- Expense-only aggregation (calculate_spent, verifying income is omitted)
- Categorized spending map (calculate_spent_by_category)
- Remaining balance calculation with negative overspent balances
- Percentage used metrics

tests/test_savings_logic.py (11 Tests)

- Overall income, expense, and net balance calculation (calculate_totals)
- Monthly trend aggregation across arbitrary date ranges (calculate_monthly_summary)
- Verification of negative monthly savings when expenses exceed income (FR-14)

tests/test_validators.py (22 Tests)

- Strict parsing and rejection of invalid / future / out-of-range dates (NFR-11)
- Validation of numeric money amounts and rejection of non-numeric, zero, or negative inputs (NFR-9)
- Category name sanitization, length caps, and whitespace trimming
- Multi-field composite validation (validate_transaction_fields)

tests/test_csv_validator.py (22 Tests)

- Validation of exact header columns: date,category,amount,type in strict order (NFR-6)
- All-or-nothing rollback policy (rejection of the entire batch if a single row has bad data)

- Unknown category detection (preventing auto-creation through import)
- Malformed row syntax, missing values, and corrupted UTF-8 decoding checks

3.2 Data Layer & Repository (Integration Tests)

tests/test_services_with_database.py (38 Tests)

- In-memory isolated database fixtures (sqlite3)
- Full CRUD lifecycle for transactions (Add, Get, Update, Delete)
- Category creation and seeding verification
- Budget limit upserting and standing limit constraints
- Transaction date range queries and sorting verification
- Foreign key integrity and cascade constraints

3.3 Frontend & Presentation (App Tests)

tests/test_app_pages.py (30 Tests)

- Headless Streamlit execution using Streamlit AppTest API
- Verification of input form widgets, select boxes, and radio toggles on: 0_Dashboard.py, 1_Transactions.py, 2_Categories.py, 3_Budgets.py, 4_Analytics.py, 5_Import_Export.py
- Form clearing on successful submit via st.session_state versioning
- Error banner display upon validation failure (NFR-3)

3.4 System & Performance Verification

tests/test_performance.py (6 Tests)

- Verification of NFR-1: rendering 500 stored transactions in < 2.0 seconds
- Verification of NFR-13: CRUD operations responding in < 2.0 seconds
- Batch performance over large synthetic datasets

4. Requirements Traceability Matrix

Requirement	Test File	Test Case Example	Result
FR-1, FR-6	test_validators.py, test_services_with_database.py	test_add_transaction_persists_record	**Passed**
FR-2	test_services_with_database.py	test_update_transaction_modifies_record	**Passed**
FR-3	test_services_with_database.py	test_delete_transaction_removes_record	**Passed**
FR-4	test_services_with_database.py, test_app_pages.py	test_transactions_sorted_by_date	**Passed**
FR-5	test_services_with_database.py	test_add_custom_category	**Passed**
FR-7	test_services_with_database.py	test_set_budget_limit	**Passed**
FR-8, FR-9	test_budget_calculator.py	test_calculate_remaining_and_alerts	**Passed**
FR-10, FR-11	test_csv_validator.py	test_validate_valid_csv_batch	**Passed**
FR-12, FR-13, FR-14	test_savings_logic.py, test_app_pages.py	test_savings_trend_three_series	**Passed**
FR-15	test_savings_logic.py	test_totals_income_expense_net	**Passed**
FR-16, NFR-4	test_services_with_database.py	test_database_persistence_across_connections	**Passed**
NFR-1, NFR-13	test_performance.py	test_transactions_page_renders_within_two_seconds	**Passed**
NFR-5, NFR-9, NFR-11	test_validators.py	test_reject_negative_amount_and_invalid_date	**Passed**
NFR-6	test_csv_validator.py	test_reject_csv_missing_columns_all_or_nothing	**Passed**
NFR-7	Architecture separation	Modular verification (Services do not import Streamlit / SQL)	**Passed**
NFR-12	test_services_with_database.py	test_database_error_handling	**Passed**